

SAE 302

Yanis Dezzaz
Guillaume Greder
Mathis Guesdon

Table des matières

1. Objectif de la SAE.....	1
2. Éléments stockés en mémoire.....	1
3. Protocole applicatif.....	2
3.1. Types de requêtes.....	2
3.2. Détail de la requête SEND.....	2
3.3. Exemple de mise en œuvre du protocole.....	3
4. La pile logicielle.....	4
5. L'interface graphique.....	5

1. Objectif de la SAE

Le but de la SAE était de pouvoir développer une application de chat. Pour cela, nous avons 2 clients et un serveur relais pour la retransmission des messages. Le serveur tout comme le client ont été écrits en Java et se basent sur UDP pour le transport des données. UDP est un protocole non fiable mais simple à implémenter. Faute de temps, ce fut donc cette solution qui a été retenue.

2. Éléments stockés en mémoire

Le client garde en mémoire:

- Son nom de session/ mot de passe
- Les 10 derniers messages de ses contacts
- Ses amis

Le serveur garde :

- Un objet session pour chaque utilisateur contenant:
 - Messages en attente/ Amis
 - Id/ mot de passe
 - Adresse IP/ port

3. Protocole applicatif

3.1. Types de requêtes

- NOTIFY :
 - Permet de signaler une erreur à la connexion ou que la boîte de réception est vide.
 - Format du paquet : NOTIFY, code
- UPDATE :
 - Permet de récupérer les messages en attentes sur le serveur
 - Format du paquet : UPDATE, mdp, login, code
- SEND :
 - Envoi d'un message à un autre utilisateur
 - Format du paquet : SEND, mdp, from, to, date, content
- FORWARD :
 - Transfère au client les messages en attentes sur le serveur
 - Format du paquet : FORWARD, from, date, content

3.2. Détail de la requête SEND

La requête SEND permet à la fois d'envoyer des messages textuels et des demandes/ réponses d'amis. Son champ content est remplie par les code suivant selon la requête souhaité:

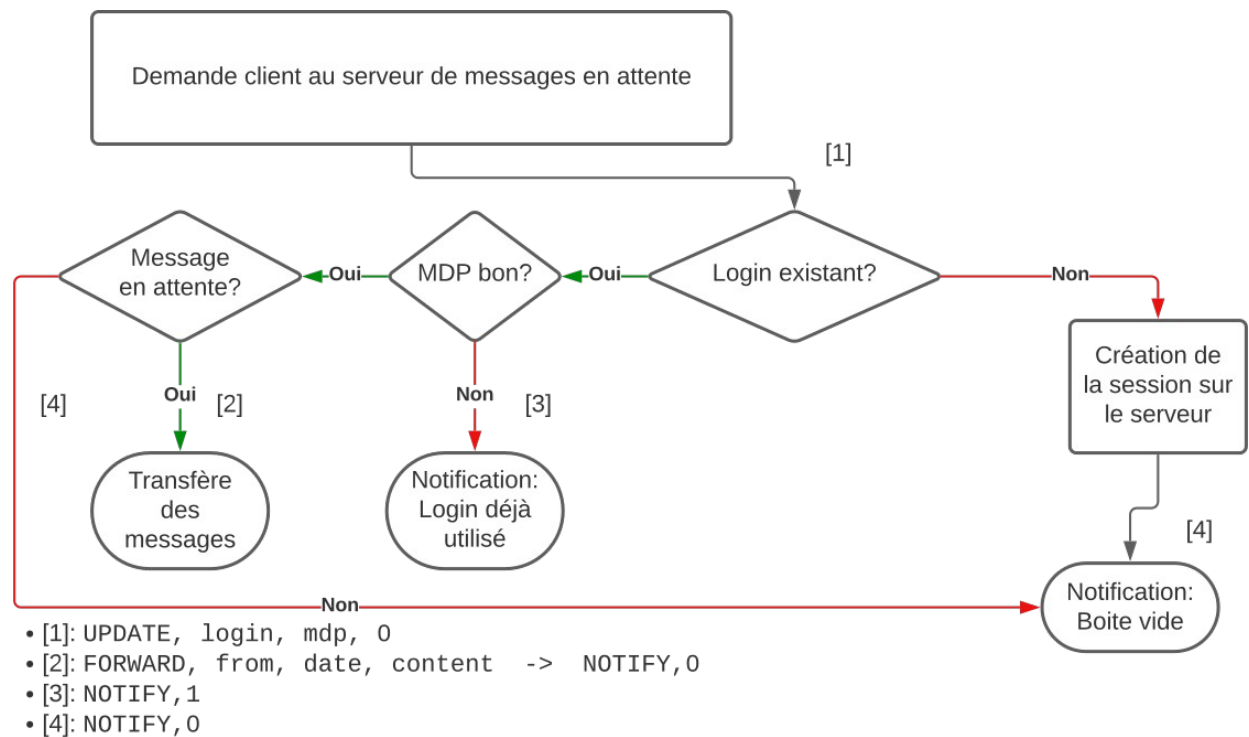
- /rqstFrd : Demande d'ami
- /accptFrd : Réponse à la demande d'ami

La procédure pour l'ajout d'un ami est la suivante:

- Une requêtes SEND avec le champ content égal à /rqstFrd est envoyé au destinataire
- Si le destinataire accepte la demande, alors ce dernier envoi une requêtes SEND avec le champ content égal à /accptFrd à l'expéditeur

Le serveur garde une trace de ces échanges puisqu'ils permettent d'ouvrir un canal de communication entre les deux lorsque ces derniers sont mutuellement amis. Sinon les messages sont filtrés et rien ne passe.

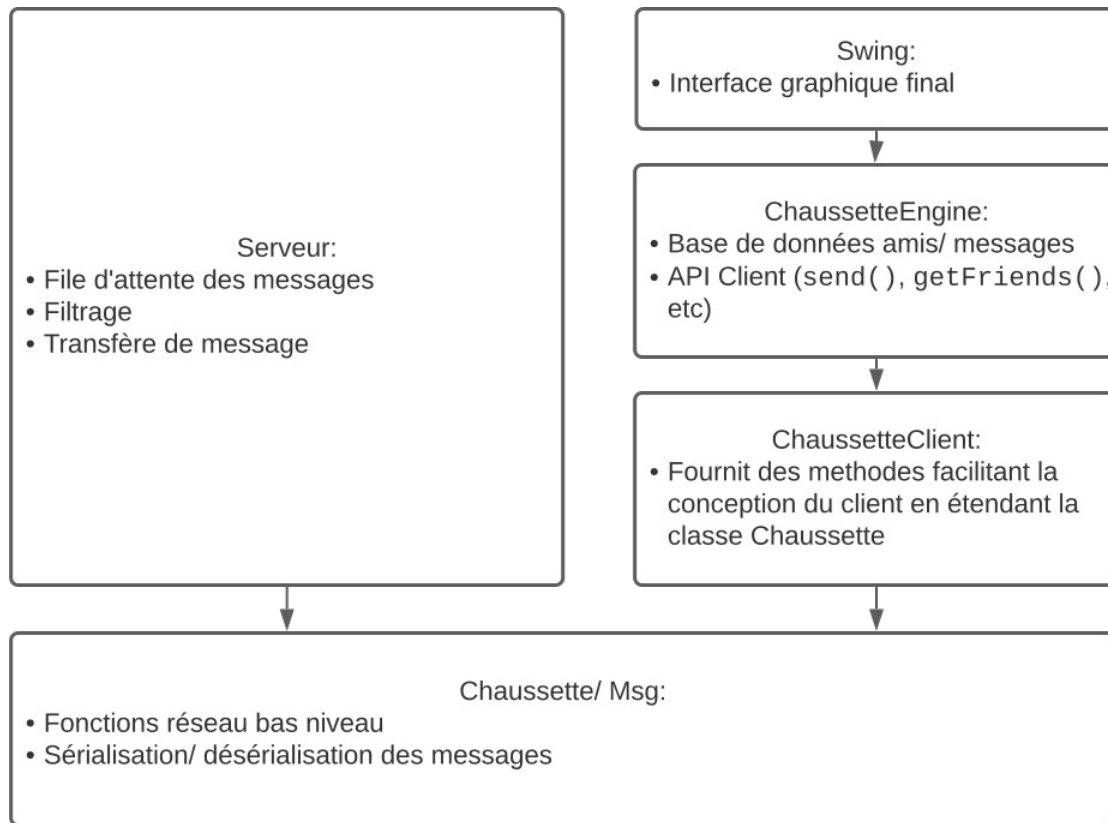
3.3. Exemple de mise en œuvre du protocole



```
Reçu:
  Message :  UPDATE,pif,pif,0
  IP       :  /127.0.0.1:52076
Envoyé:
  Message :  FORWARD,plouf,15/12/2023 11:41:59,Hello!
  IP       :  /127.0.0.1:52076
Envoyé:
  Message :  FORWARD,plouf,15/12/2023 11:41:59,Prends du pain ce midi
  IP       :  /127.0.0.1:52076
Envoyé:
  Message :  FORWARD,plouf,15/12/2023 11:41:59,Et une pâtisserie aussi!
  IP       :  /127.0.0.1:52076
Envoyé:
  Message :  NOTIFY,0
  IP       :  /127.0.0.1:52076
Reçu:
  Message :  SEND,pif,pif,plouf,15/12/2023 11:41:59,Bien sur!
  IP       :  /127.0.0.1:52076
```

Dans l'extrait ci-dessus des logs du serveur, un premier client demande ses messages en attentes. Le mot de passe (pif) et son id (pif aussi) étant correcte vis à vis de ce qui est enregistré sur le serveur, ce dernier lui transfère ses messages en attentes (FORWARD) et lui indique que la transmission est fini (NOTIFY 0: boîte vide) à la fin. Ensuite le client pif envoie à plouf le message *Bien sur!* via la requête SEND qui sera stocké sur le serveur en attendant un UPDATE de plouf. Tout ce se passe dans le cas où pif ou plouf sont amis. Sinon aucune communication serait possible entre les deux.

4. La pile logicielle



L'application a été divisée en plusieurs composants afin de pouvoir répartir les tâches dans le groupe, faciliter le débogage et éviter la redondance de code.

Les classes suivantes ont ainsi été créées:

- **Msg:** Création et manipulation des messages via des méthodes comme `getContent()`, `getDate()`, etc...
- **Session:** Création et manipulation des informations d'utilisateurs côté serveur. Elle permet l'enregistrement des messages en attentes, requêtes d'amis, IP/ port du client, etc...
- **Chaussette:** Interface pour l'envoi et la réception des messages sur UDP. Elle offre les méthodes `listen()` retournant un objet `Msg` et `send()` pour l'envoi de message.
- **ChaussetteClient:** Une surcouche à la classe `Chaussette` pour le client permettant l'envoi ainsi que la réception de message de manière simplifiée. Elle offre des méthodes retournant des listes de messages suite à une requête `UPDATE` et d'autres facilités.
- **ChaussetteEngine:** Une classe exposant des méthodes pour obtenir nos amis, envoyer des messages, mettre à jour les messages de manière transparente, etc...

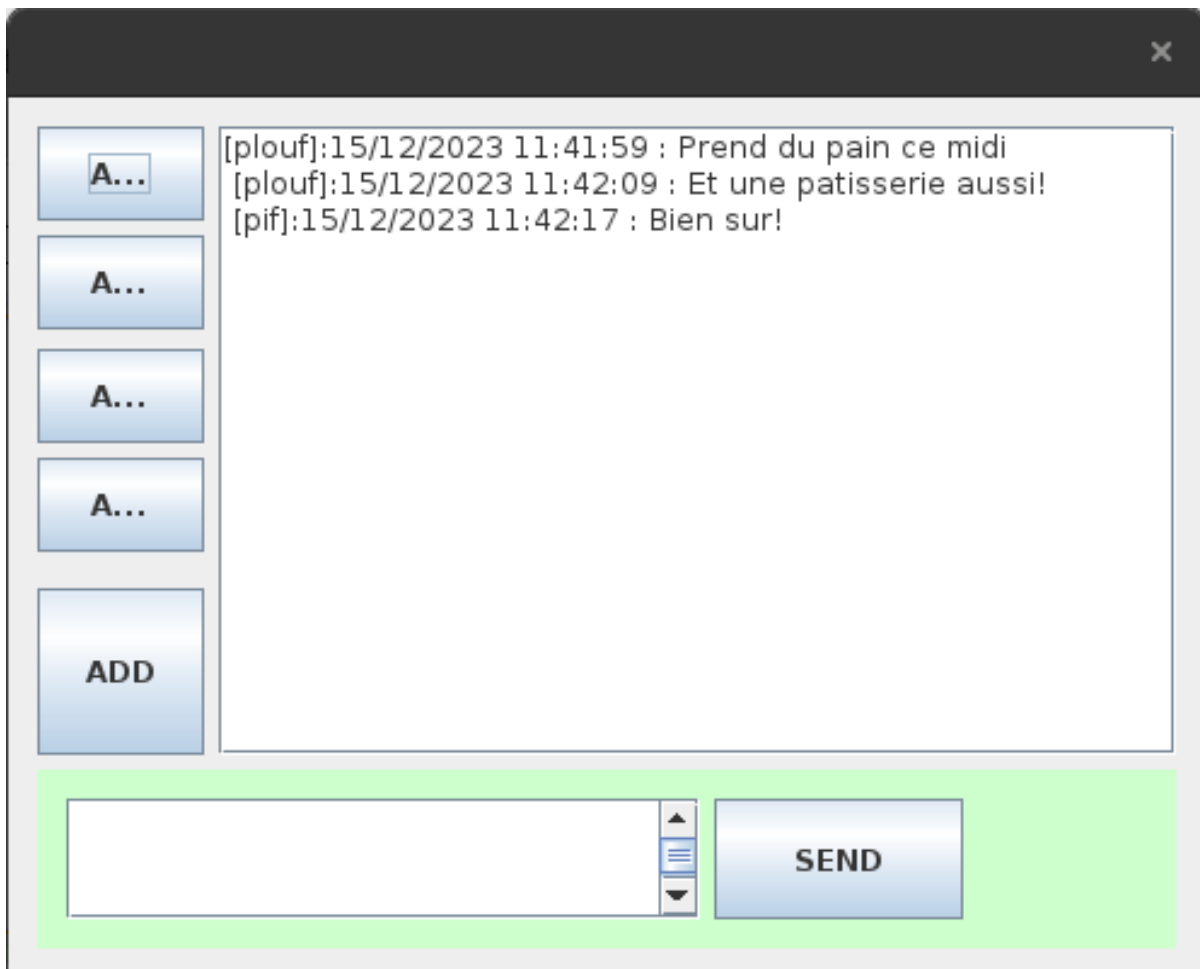
`Chaussette` et `Msg` sont des classes importantes partagées entre le client et le serveur. `ChaussetteClient` et `ChaussetteEngine` sont quant à eux spécifiques au client et `Session` au serveur.

5. L'interface graphique

Le choix du framework Swing pour la conception de l'interface graphique s'est faite pour des raisons de facilité et de compatibilité avec notre matériel, nos ordinateur ayant du mal avec Android Studio...

Notre interface graphique se compose de 3 parties:

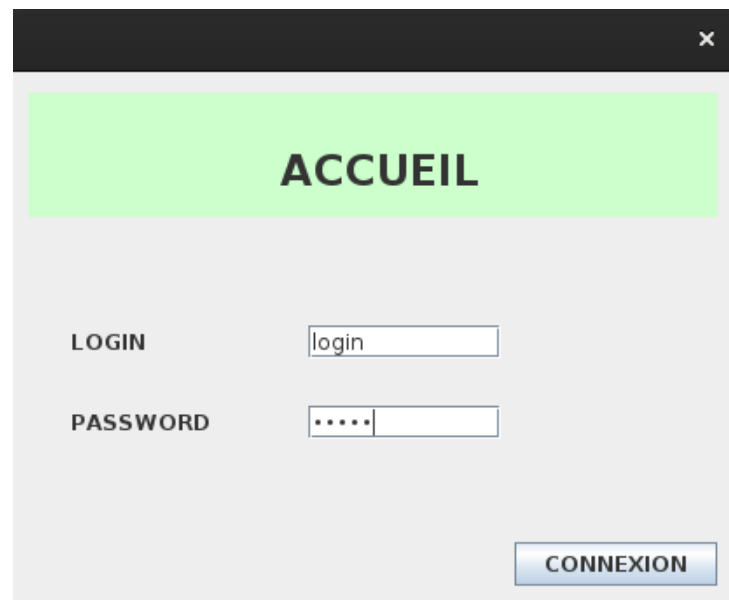
- La page de connexion
- Le chat
- La page d'ajout d'amis



La connexion se fait par la première page affichée par le logiciel. Les valeurs saisies dans les champs login et password sont stockés dans un objet ChaussetteEngine et automatiquement réutilisé par la suite.

Pour communiquer avec un autre utilisateur, il faut au préalable l'ajouter via le bouton ADD. Une requête SEND /rqstFrd est transmise au destinataire lors de la validation du login dans le champ LOGIN AMI.

Enfin le bouton SEND récupère le texte dans la zone de texte à sa gauche et l'affiche. Si aucun caractère n'est renseigné dans la zone de texte, un message d'erreur apparaîtra.



ACCUEIL

LOGIN

PASSWORD

CONNEXION



AJOUT

LOGIN AMI

AJOUTER